1. Objetivo

O objetivo do projeto é desenvolver funções de sincronização para organizar o embarque de passageiros nos carros do metrô, garantindo que os passageiros aguardem a chegada de um carro e só embarquem quando houver assentos disponíveis. O sistema deve assegurar que o carro aguarde o embarque dos passageiros esperados, respeitando sua capacidade. A solução deve ser desenvolvida em C, utilizando threads, mutexes e variáveis de condição, sem semáforos ou busy-wait. Além disso, a compilação deve ser realizada por meio de um Makefile, gerando o executável sync_car.

2. Checklist de Requisitos

Requisito do enunciado	Status	Como testar / evidência
Implementar o projeto em linguagem C	OK	Conferir os arquivos sync_car.c e sync_car.h.
Definir a estrutura station	OK	Verificar a estrutura declarada em sync_car.h
Implementadas todas as funções solicitadas no enunciado: station_init(), station_load_car(), station_wait_for_car() e station_on_board()	OK	Executar make para compilar o programa e, em seguida, ./sync_car para testar seu funcionamento
Utilizar mutex para proteger os dados compartilhados	OK	Conferir as operações de bloqueio e liberação do mutex no código.
Utilizar variáveis de condição para sincronização	OK	Verificar o uso de pthread_cond_wait(), pthread_cond_signal() e pthread_cond_broadcast().
Utilizar no máximo um mutex por estação	OK	Conferir a quantidade de mutexes na estrutura station.
Não utilizar semáforos	OK	Inspecionar os arquivos de implementação e verificar se não há semáforos.
Evitar busy-wait	OK	Verificar se as threads aguardam usando

		pthread_cond_wait().
Permitir o retorno de vários passageiros antes das confirmações de embarque	OK	Verificar se vários passageiros conseguem retornar de station_wait_for_car() antes de chamarem station_on_board().
Respeitar a capacidade do carro	OK	Executar ./sync_car e conferir os passageiros embarcados em relação às vagas disponíveis.
Aguardar as confirmações de embarque antes de liberar o carro	OK	Executar o runner e verificar se não ocorre o erro station_load_car returned early!.
Tratar a chegada do carro sem vagas disponíveis	OK e testado	Verificar as chamadas com zero vagas e confirmar que retornam imediatamente.
Permitir que o carro parta quando todos os passageiros que esperavam embarcarem	OK e testado	Testar um carro com mais assentos do que passageiros esperando.
Compilar por meio de um Makefile	OK	Executar make clean e make
Gerar o executável sync_car	OK	Executar ls -l sync_car após a compilação.
Disponibilizar instruções de compilação e execução	OK	Conferir o README.md.
Registrar os testes realizados	OK	Conferir o arquivo evidencias.log e os registros das execuções.
Entregar o relatório em PDF	OK	Conferir o relatório conforme o Guia de Relatórios.
Disponibilizar o código, README e relatório no GitHub	OK	Conferir os arquivos presentes no repositório.

3. Como Reproduzir

Para reproduzir o projeto, acesse a pasta raiz do repositório e execute:

-make clean e make

Para limpar arquivos anteriores e compilar o programa.

Em seguida, execute:

```
./sync_car
```

para iniciar os testes.

4. Arquitetura

Arquivos e responsabilidades:

1. sync_car.h - Define a estrutura station, suas variáveis de sincronização e os protótipos das funções.
2. sync_car.c - Implementa as funções de inicialização da estação, espera pelo carro, autorização de embarque e confirmação dos passageiros.
3. sync_car-runner.c - Executa os testes do programa, simulando a chegada dos carros e o embarque dos passageiros.
4. Makefile - Automatiza a compilação do projeto e a remoção do executável gerado.

Decisões técnicas:

1. Utilização de mutex - Proteger o acesso às variáveis compartilhadas - Evitar condições de corrida entre as threads.
2. Utilização de variáveis de condição - Sincronizar a chegada do carro e o embarque dos passageiros - Permitir que as threads aguardem os eventos necessários sem busy-wait.
3. Utilização de pthreads - Executar passageiros de forma concorrente - Permitir o embarque simultâneo respeitando a capacidade do carro.

5. Estratégias e Diário de Desenvolvimento

Isto é o núcleo do relatório.

5.1 Estratégias da semana

Estratégia	**Nome curto**	**Contexto**	**Motivo da troca (se houve)**
S1	Sincronização com pthreads.	Implementação das funções utilizando mutex e variáveis de condição para controlar o embarque dos passageiros.	Não houve troca de estratégia.

5.2 Diário de Tentativas .

#	**Estrat.**	**O que tentei**	**Resultado**	**Hipótese/Causa (se falhou)**	**Quando**	**Evidência**
1	S1	Implementar a estrutura station e inicializar mutex, variáveis de condição e contadores.	SUCESSO		28/10/2026 - durante o desenvolvimento	Código em sync_car.c e sync_car.h.
2	S1	Implementar	SUCESSO		28/10/2026 -	Implementaç

		station_wait_for_car() para aguardar a chegada do carro e a disponibilidade de assentos.			durante o desenvolvimento	ão da função em sync_car.c.
3	S1	Implementar station_load_car() para liberar o embarque e aguardar a confirmação dos passageiros.	RESULTADO PARCIAL	A função retornava antes de todos os passageiros previstos confirmarem o embarque.	Durante os testes	Teste do runner que indicou retorno antecipado.
4	S1	Adicionar uma espera com variável de condição até a confirmação do embarque previsto.	SUCESSO		28/10/2026 - Após identificar a falha	Trecho atualizado de station_load_car().
5	S1	Repetir a execução do programa dez vezes para verificar sua estabilidade.	SUCESSO		28/10/2026 - durante os testes	Dez execuções adicionais concluídas com Looks good!.

6. Evidências

6.1 Log automático (faz o trabalho pesado por você)

Para registrar as etapas do desenvolvimento, utilizei o comando script -a evidencias.log, que grava automaticamente os comandos executados e as saídas do terminal. O arquivo evidencias.log será incluído no arquivo .tar da entrega.

6.2 Prints

- Erro principal: captura do teste em que a função station_load_car() retornava antes da confirmação do embarque dos passageiros.
- Validação final: captura do terminal mostrando a compilação com make e a execução bem-sucedida do programa, com a mensagem Looks good!.

Print do erro:

```
luisf@aluisio:/mnt/c/Users/luisf/FACULDADE/INFRA DE SOFT/sincronizacao-metro/sincronizacao-metro$ gcc -Wall -Wextra -pthread sync_car.c sync
_car-runner.c -o sync_car
sync_car-runner.c: In function ‘alarm_handler’:
sync_car-runner.c:54:24: warning: unused parameter ‘foo’ [-Wunused-parameter]
   54 | void alarm_handler(int foo)
      |                    ~~~~^~~
luisf@aluisio:/mnt/c/Users/luisf/FACULDADE/INFRA DE SOFT/sincronizacao-metro/sincronizacao-metro$ ./sync_car
car entering station with 37 free seats
car departed station with 100 new passenger(s) (expected 37) *****
Error: Many passengers on this car
luisf@aluisio:/mnt/c/Users/luisf/FACULDADE/INFRA DE SOFT/sincronizacao-metro/sincronizacao-metro$
```

O erro estava relacionado à sincronização das threads usando variáveis de condição. Inicialmente, utilizei um if para verificar a disponibilidade de vagas. Porém, quando uma thread era acordada pelo pthread_cond_broadcast(), ela não verificava novamente se a condição continuava válida. Isso poderia permitir que uma thread prosseguisse em uma situação inadequada.

Para corrigir esse comportamento, substituí o if por um while, garantindo que a condição fosse reavaliada após cada despertar. Dessa forma, a thread só continua quando a condição necessária é verdadeira, mantendo a sincronização entre os passageiros e o carro.

Teste passando depois da correção:

```
luisf@aluisio:/mnt/c/Users/luisf/FACULDADE/INFRA DE SOFT/sincronizacao-metro/sincronizacao-metro$ ./sync_car
car entering station with 44 free seats
car departed station with 44 new passenger(s) (expected 44)
car entering station with 49 free seats
car departed station with 49 new passenger(s) (expected 49)
car entering station with 20 free seats
car departed station with 7 new passenger(s) (expected 7)
Looks good!
```

7. Uso de IA

Utilizei a IA principalmente como ferramenta de aprendizado, pois inicialmente não conhecia bem os conceitos envolvidos no projeto. Ela me ajudou a compreender os conteúdos, organizar as etapas e testar a implementação, atuando como assistente durante o desenvolvimento, sem fornecer o código pronto.

Prompts principais:

- Explique conceitos de sincronização e concorrência.
- Me diga como funcionam mutexes e semáforos.
- Me ajude a entender melhor cada função que é obrigatório usar no enunciado do professor.
- acho que o erro tá na condição do if, que poderia estar controlando incorretamente quando o carro deveria encerrar o embarque, o uso do if nessa condição ta errado?

O que validei manualmente e como:

Validei a implementação compilando o projeto com make clean e make e executando ./sync_car. Também repeti a execução mais de dez vezes, verificando as mensagens do terminal, a quantidade de passageiros embarcados em comparação com a esperada pelo runner e a conclusão dos testes com a mensagem Looks good!.

8. Reflexão Final

Durante o projeto, aprendi a importância da sincronização entre threads e do uso de mutexes e variáveis de condição. Escolhi essas ferramentas para controlar o acesso aos dados compartilhados e organizar o embarque dos passageiros. Tive dificuldades para entender a lógica das condições e garantir que o carro aguardasse as confirmações necessárias. Com os testes, consegui identificar e corrigir os problemas encontrados. Com mais tempo, melhoraria a cobertura dos testes para verificar outros cenários. Na próxima vez, pretendo testar cada função separadamente desde o início. Também evitaria tentar resolver tudo de uma vez, dividindo o desenvolvimento em etapas menores.

9. Checklist Final de Entrega

Item	Confirmado?
Compilei do zero seguindo só o que está no relatório	OK
Rodei os testes e evidencias.log foi gerado	OK
Tenho prints obrigatórios	OK
Testei pelo menos um caso limite e um caso inválido	OK
Preenchi a seção de uso de IA	OK
Revisei o relatório e removi frases genéricas / vazias	OK

10. Se eu tivesse mais 2 horas

Se tivesse mais duas horas, ampliaria os testes para verificar diferentes quantidades de passageiros e assentos disponíveis. Também testaria situações com zero assentos e com menos passageiros esperando do que a capacidade do carro. Revisaria as condições de espera para reduzir o risco de problemas de sincronização. Verificaria possíveis warnings durante a compilação e corrigiria os que fossem causados pelo meu código. Melhoraria a organização dos comentários para facilitar a compreensão das funções. Conferiria se todos os recursos de sincronização são inicializados corretamente. Por fim, registraria os resultados dos testes no evidencias.log e verificaria novamente a compilação pelo Makefile.